



UNIVERSITI TEKNOLOGI MALAYSIA

**PROGRAMMING TECHNIQUE II
(SECJ1023)**

SEMESTER 2 2024/2025

GROUP PROJECT

E-Voting System

MEMBER 1'S YEOH KENG WEI (A24CS0316)

MEMBER 2'S EDWIN OO MING HAO (A24CS0245)

MEMBER 3'S CHING QI YAN (A24CS0235)

1 SECRH

SECTION 04

Lecturer:

MADAM LIZAWATI MI YUSUF

19th JUNE 2025

Table of Content

1. SECTION A: PROJECT DESCRIPTION.....	2
Synopsis	2
Objective and Scope	2
Work flow	3
OO Concepts	4
2. SECTION B: CLASS DIAGRAMS	6
Class User	7
Class Admin.....	7
Class Candidate.....	8
Class Voter.....	8
Class Election.....	9
Class FileManager	9
3. SECTION C: SOURCE CODE AND USER MANUAL.....	10
Main Screen	10
Admin User Manual.....	13
Voter User Manual.....	21
4. Appendix.....	27

SECTION A: PROJECT DESCRIPTION

Synopsis

Design a system to simulate a secure and transparent digital voting process for environments such as student councils, clubs, or even food or animal. This system supports role-based access for three main user types: Admin, Voter, and Candidate. Each role has specific functions:

- *Admin manages the entire election process, including registering candidates and voters, starting and closing the election, and viewing final results including history result.*
- *Voters register and log in, view candidate details, and cast a vote. Each voter is allowed to vote only once.*
- *Candidates can log in to view and update their campaign details.*
- *If candidates is not a person, the admin will access to update the details*

Objective and Scope

The goal of this system is to provide a simple, secure, and fair digital voting experience. It is designed for use in environments like student elections or club voting, where transparency and one-person-one-vote enforcement are important.

Work flow

Figure below show the flowchart about how the system loop and end. The system using login function and will check through all users function then go through the class and loop until user exit and system will clear the interface to prevent information leak to next user when login. Figure 1.1 below shows the flow chart of our system.

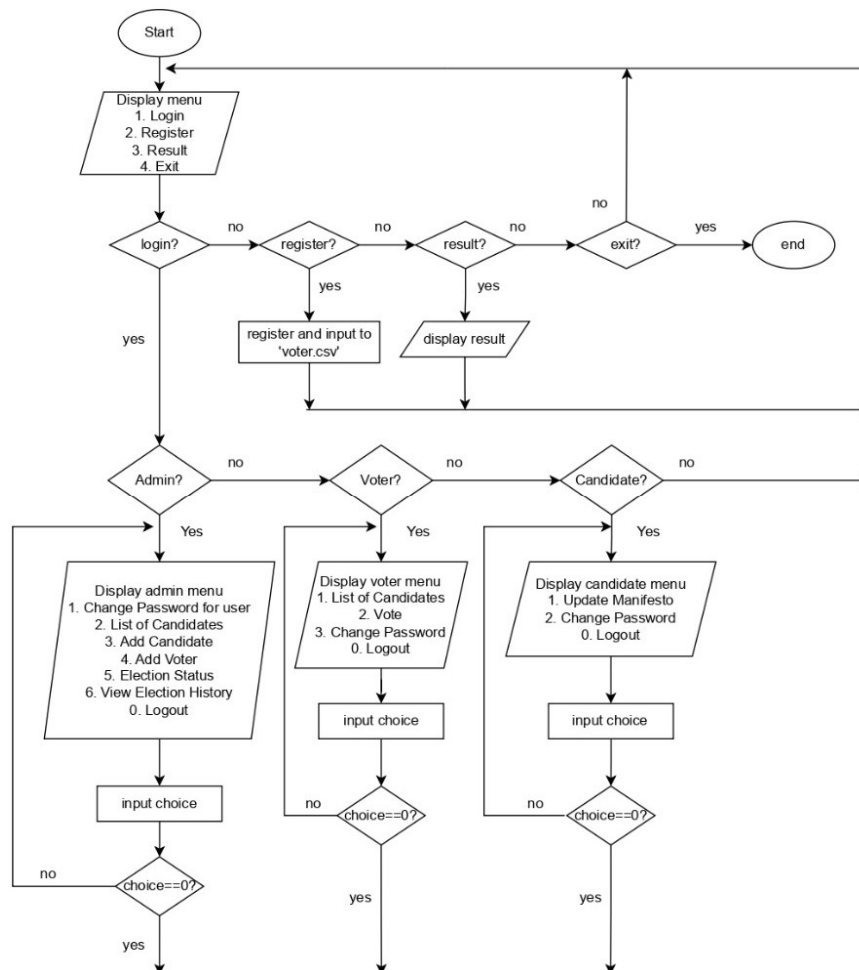


Figure 1.1

OO Concepts

1. Inheritance

User (Base class) -> Admin, Voter, Candidate (Derived class)

User is abstract base class with common attributes: userID, name, password (protector member)

The benefits are consistent interface across different user types and extensibility for adding new user types.

```
1 Voter.hpp > Voter
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5 #include <iomanip>
6 #include "FileManager.hpp"
7
8 #ifndef VOTER_H
9 #define VOTER_H
10 class Voter : public User{
11 private:
12     bool hasVoted;
13     bool needsUpdate;
14     Admin *admin;
```

Class voter

```
1 Admin.hpp > Admin
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5 #include <iomanip>
6 #include "FileManager.hpp"
7 #include "Election.hpp"
8
9 #ifndef ADMIN_H
10 #define ADMIN_H
11 class Admin : public User{
12 private:
13     Election election;
```

Class admin

```
1 Candidate.hpp > Candidate
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5 #include <iomanip>
6 #include "FileManager.hpp"
7
8 #ifndef CANDIDATE_H
9 #define CANDIDATE_H
10 class Candidate : public User{
11 private:
12     string manifesto;
13     int voteCount;
14     bool needsUpdate;
```

Class candidate

2. Polymorphism

Each derived class has its own implementation of the function login():

Admin, Voter and Candidate after login() it will all call their own Menu page inside the class.

```
1 User.hpp > User > login()
2 #ifndef USER_H
3 #define USER_H
4
5 class User{
6 protected:
7     string userID;
8     string name;
9     string password;
10
11 public:
12     User(string_userID="", string_name="", string_password="")
13     : userID(_userID), name(_name), password(_password){}
14
15     virtual void login()=0;
16     virtual ~User(){}
17
18     void updatePassword(const string& filename, const string& tempF
19     string oldPassword, newPassword;
20     cout<<"\n== Change Password ==\n";
21     cout<<"Enter Current Password: ";
```

Class user function login() is a virtual function and each child class will override the function.

3. Association (Composition)

Admin has-an Election

For example, Election has startElection() the Admin will need to call the function then only it can work.

```
Admin.hpp > Admin > election
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <iomanip>
5 #include "FileManager.hpp"
6 #include "Election.hpp"
7
8 #ifndef ADMIN_H
9 #define ADMIN_H
10 class Admin : public User{
11 private:
12     Election election;
13
14 public:
15     Admin(string _userID="", string _name="", string _password="")
16         : User(_userID, _name, _password){}
17
18     void login() override{
19         cout<<"\nWelcome, Admin: "<<name<<"\n";
20         adminMenu();
21     }
22 }
```

Declare a "Election election" inside class Admin and it act as composition.

4. Association (Aggregation)

Voter uses-an Admin

For example, the voter can check the list of candidates which already implement in admin and the voter can use it.

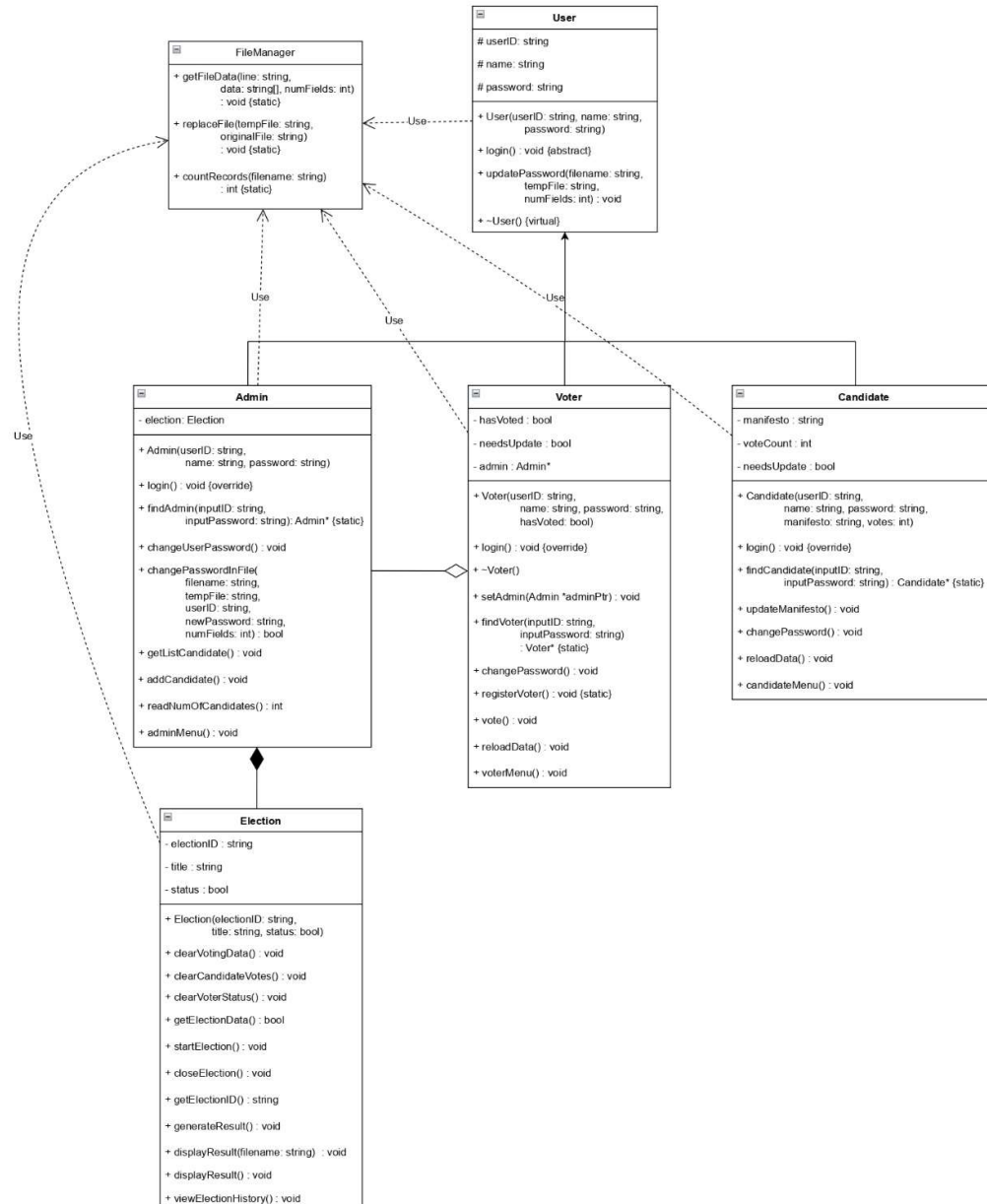
```
Voter.hpp > Voter > admin
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <iomanip>
5 #include "FileManager.hpp"
6
7 #ifndef VOTER_H
8 #define VOTER_H
9 class Voter : public User{
10 private:
11     bool hasVoted;
12     bool needsUpdate;
13     Admin *admin;
14 }
```

Using a pointer on admin and voter will use the function of admin on getting candidate list.

```
Voter.hpp > Voter > vote()
7 #ifndef VOTER_H
9 class Voter : public User{
92     void vote(){
93
94
95
96
97
98
99
100         if(hasVoted){
101             cout<<"You have already voted.\n";
102             return;
103         }
104
105         int numCandidates=admin->readNumOfCandidates();
106         int choice;
107         admin->getListCandidate();
108     }
```

SECTION B: CLASS DIAGRAMS

Diagram below shows the UML of our system and shows all object-oriented concept in our system.



The work in this page has been done by:
EDWIN OO MING HAO

Date: **19/6/2025**

Class User

Attributes	Description
userID	Unique identifier for each user
name	Full name of the user
password	Password for user authentication
Methods	Description
User()	Constructor to initialize user with ID, name, and password
login()	Pure virtual method that must be implemented by derived classes
updatePassword()	Updates user password by verifying old password and setting new one
~User()	Virtual destructor for proper cleanup

Class Admin

Attributes	Description
election	Election object to manage election operations
Methods	Description
Admin()	Constructor to initialize admin with user credentials
login()	Displays welcome message and shows admin menu
findAdmin()	Static method to authenticate admin login credentials from admin.csv
changeUserPassword()	Allows admin to change password for any voter or candidate
changePasswordInFile()	Helper method to update password in specific CSV file
getListCandidate()	Displays formatted list of all candidates with their manifestos
addCandidate()	Adds new candidate to the system (only when no election is ongoing)
readNumOfCandidates()	Returns the total number of candidates in the system
adminMenu()	Displays admin menu with options for election management

Class Candidate

Attributes	Description
manifesto	Campaign manifesto/statement of the candidate
voteCount	Number of votes received by the candidate
needsUpdate	Flag to track if candidate data needs to be updated
Methods	Description
Candidate()	Constructor to initialize candidate with credentials, manifesto, and vote count
login()	Displays candidate menu upon successful login
findCandidate()	Static method to authenticate candidate login from candidates.csv
updateManifesto()	Allows candidate to update their campaign manifesto
changePassword()	Allows candidate to change their password
reloadData()	Updates candidate data in CSV file and refreshes local data
candidateMenu()	Displays candidate menu with options to update manifesto and change password

Class Voter

Attributes	Description
hasVoted	Boolean flag indicating if voter has cast their vote
needsUpdate	Flag to track if voter data needs to be updated
admin	Pointer to Admin object for accessing admin functionalities
Methods	Description
Voter()	Constructor to initialize voter with credentials and voting status
login()	Displays voter menu upon successful login
~Voter()	Destructor to clean up admin pointer
setAdmin()	Sets the admin pointer for accessing admin functions
findVoter()	Static method to authenticate voter login from voters.csv
changePassword()	Allows voter to change their password
registerVoter()	Static method for new voter registration
vote()	Allows voter to cast their vote for a candidate
reloadData()	Updates voter data in CSV file and refreshes local data
voterMenu()	Displays voter menu with voting and profile management options

The work in this page has been done by:
EDWIN OO MING HAO

Date: **19/6/2025**

Class Election

Attributes	Description
electionID	Unique identifier for each election
title	The title or name of an election
status	Boolean indicating if election is ongoing or ended
Methods	Description
Election()	Constructor to initialize election with ID, title, and status
clearVotingData()	Clears all voting data after election ends
clearCandidateVotes()	Resets all candidate vote counts to zero
clearVoterStatus()	Resets all voter voting status to false
getElectionData()	Retrieves current election status and data
startElection()	Starts a new election with specified title
closeElection()	Ends the ongoing election and generates results
getElectionID()	Returns the ID of the most recent election
generateResult()	Generates and saves election results to file
displayResult()	Displays election results from result file
viewElectionHistory()	Shows history of all elections and their results

Class FileManager

Attributes	Description
-	Static utility class with no instance attributes
Methods	Description
getFileData()	Parses CSV line and extracts data into array
replaceFile()	Replaces original file with temporary file
countRecords()	Counts number of records in a CSV file (excluding header)

SECTION C: SOURCE CODE AND USER MANUAL

Input of user is indicated by a blue rectangle. Etc. 

Main Screen

```
=== E-Voting System ===  
1. Login  
2. Register as Voter  
3. Display Result  
0. Exit  
Enter your choice: |
```

This is the welcome screen shown when the program is run, the first screen may be seen. Choose

- 1 to log in into user's account, the account type can be admin, candidates or voter.
- 2 to register user as a voter. User can only register if the admin add user's username and full name into the system.
- 3 to display result of ended election.
- Choose 0 to exit the program, the program will be terminated.

1. When User input "1" to login

```
=== E-Voting System ===  
1. Login  
2. Register as Voter  
3. Display Result  
0. Exit  
Enter your choice: 1  
  
Enter Username: abc  
Enter Password: pass.
```

Users will be asked to input their Username and Password and will be auto assigned to their own menu.

- [Admin User Manual](#)
- [Voter User Manual](#)
- [Candidate User Manual](#)

2. When User input "2" for Register as Voter

```
=== E-Voting System ===
1. Login
2. Register as Voter
3. Display Result
0. Exit
Enter your choice: 2

=== Voter Registration ===
Enter your Username: john
Enter your Full Name: Johnson Koh
Set your Password: nice
Registration successful!
Press any key to continue . . .
```

This register is only for voters and with the name list that will be provided by admin, which they will need to input Username and Full Name based on what admin input in voter.csv file, and each voter will receive their Username confidentially.

3. When User input "3" for Display Result

```
=== E-Voting System ===
1. Login
2. Register as Voter
3. Display Result
0. Exit
Enter your choice: 3

=== ELECTION RESULT ===
Title: Testing
Highest votes of candidate: 'Jason'
Ranking||Candidates' Name    ||Number of Votes
1      ||Jason                    ||15
2      ||Alan                      ||10
3      ||Ryan                      ||5
Press any key to continue . . .
```

The system will output the latest ended election on the screen, and all users are not allowed to view on the ongoing election result as it may affect choice to vote for by voters.

4. When user input “0” to exit program. The program will execute and close

```
=== E-Voting System ===  
1. Login  
2. Register as Voter  
3. Display Result  
0. Exit  
Enter your choice: 0  
  
Exiting system...  
Press any key to continue . . .
```

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

Admin User Manual

After successful input Admin's username and password, the system will show the output below.

Screen Admin Menu:

```
Welcome, Admin: a

--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: |
```

Choose

- 1 to change password for user, can be both voters or candidates.
- 2 to see the list of candidates.
- 3 to add candidates into system.
- 4 to add voters to the system.
- 5 to check election status and start or stop an election.
- 6 to view the election history, all election organize can be seen.
- 0 to logout, program will return to the main screen.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

1. Change password for user

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 1
```

Enter '1' at "Enter your choice:". The screen **Change Password for User** will be shown.

Screen Change Password for User:

```
=== Change Password for User ===
Enter Username: ching
Set New Password: 0123abc
Password changed for CHINGQIYAN successfully.
```

Enter username of account desired to change password. Enter the new password. If the username is recorded in system the password is changed successfully. Admin can change password for both candidates and voter.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

2. See list of candidates

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 2
```

Enter '2' at "Enter your choice:". The screen **List of Candidate** will be shown.

Screen List of Candidate:

```
-----LIST OF CANDIDATES-----
=====
No    | Name                | Manifesto
1     | Edwin Oo           | vote me!
2     | Muhammad Ali        | I am the one
3     | Mickey Trump        | Make UTM great again
4     | Nashwaran           | Bring UTM glorious again!!!
```

The list of number give to the candidates and name of the candidates and their manifesto is shown.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

3. Add candidate into the system

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 3
```

Enter '3' at "Enter your choice:". The screen **Add new Candidate** will be shown.

Screen Add new Candidate:

```
=== Add New Candidate ===
Enter Candidate Username: John
Enter Candidate Full Name: John Yeoh
Set Candidate Password: ykwei05
Candidate 'John Yeoh' added successfully.
Note: Candidate can login to add their manifesto.
```

Enter the username of the new candidate and the full name of candidate. Then, set the password for the candidate.

```
-----LIST OF CANDIDATES-----
=====
No  | Name                | Manifesto
1   | Edwin Oo            | vote me!
2   | Muhammad Ali         | I am the one
3   | Mickey Trump        | Make UTM great again
4   | Nashwaran           | Bring UTM glorious again!!!
5   | John Yeoh           | NULL
```

The list of candidates will be shown automatically for checking purposes. The manifesto of the new candidate is set to NULL by default. The candidate can login to add their manifesto.

```
Do you want to add another candidate? (Y/N):
```

After that the system will ask whether to add another candidate or not. Enter 'Y', screen **Add new Candidate** will be shown. Another candidate can be added to the system. Enter 'N' to return screen **Admin Menu**.

Date: 19/6/2025

The work in this page has been done by: CHING QI YAN

4. Add voter into the system

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 4
```

Enter '4' at "Enter your choice:". The screen **Add new Voter** will be shown.

Screen Add new Voter:

```
=== Add New Voter ===
Enter Voter Username: ching
Enter Voter Full Name: CHINGQIYAN
Voter 'CHINGQIYAN' added successfully.
Note: Voter needs to register to set their password.
```

Enter the username and full name of voter.

```
Do you want to add another voter? (Y/N): |
```

The system will ask either to add another voter or not. Enter 'Y', the screen **Add new Voter** will be shown. Another voter can be added to the system. Enter 'N' to return screen **Admin Menu**.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

5. Start or end or check the status of election

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 5
```

Enter '5' at "Enter your choice:". The screen **Election Status** will be shown.

Screen Election Status:

```
--- Election Status ---
There is no ongoing Election, you may start a new election.
Enter the Election title: Who is the leader of the team
Enter 'Y' to confirm: y
Election 'Who is the leader of the team' has started.
```

While there is no ongoing election, a new election can be started. Enter the title of the election which is the name of election. Then, confirm to start by entering 'y'.

Screen Election Status:

```
--- Election Status ---
You cannot start a new Election.
There is already an ongoing election!
Title: Who is the leader of the team
Do you want to close this Election 'Who is the leader of the team'?
Enter 'Y' to confirm end election: y
Election 'Who is the leader of the team' has ended successfully.
```

While there is an ongoing election, admin can choose to stop the election. Enter 'y' to stop the election. The screen **Election Result** will be shown automatically

Screen Election Result:

```
=== ELECTION RESULT ===
Title: Who is the leader of the team
Highest votes of candidate: 'Edwin Oo' and 'Muhammad Ali' and 'Mickey Trump' and 'Nashwaran' and 'John Yeoh'
Ranking|Candidates' Name||Number of Votes
1||Edwin Oo||0
1||Muhammad Ali||0
1||Mickey Trump||0
1||Nashwaran||0
1||John Yeoh||0
```

The highest votes of candidates is determined by the system.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

6. View the election history

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 6
```

Enter '6' at "Enter your choice:". The screen **Election History** will be shown.

Screen Election History:

```
=== Election History ===
No  || Election ID || Title                                     || Status
=====
1   || 001         || test1                                     || Ended
2   || 002         || asdfg                                    || Ended
3   || 003         || test3                                    || Ended
4   || 004         || AHH9862                                 || Ended
5   || 005         || Who is the leader of the team           || Ended
Enter Election ID to view result (or 0 to return): 005
```

The election ID produced by the system is shown. Enter the election ID to view result.
Note: Must follow exactly the election ID.

Screen Election Result:

```
=== ELECTION RESULT ===
Title: Who is the leader of the team
Highest votes of candidate: 'Edwin Oo' and 'Muhammad Ali' and 'Mickey Trump' and 'Nashwaran' and 'John Yeoh'
Ranking|Candidates' Name|Number of Votes
1      || Edwin Oo      || 0
1      || Muhammad Ali  || 0
1      || Mickey Trump  || 0
1      || Nashwaran     || 0
1      || John Yeoh     || 0
```

The screen **Election Result** of the election selected is shown.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

7. Logout from admin account

Screen Admin Menu:

```
--- Admin Menu ---
1. Change Password for user
2. List of Candidates
3. Add Candidate
4. Add Voter
5. Election Status
6. View Election History
0. Logout
Enter your choice: 0
Logging out...
Press any key to continue . . . |
```

Enter '0' at "Enter your choice:". Then press any key to return to main screen.

The work in this page has been done by: Yeoh Keng Wei Date: 19/6/2025

Voter User Manual

After successful input Voter's username and password, the system will show the output below.

Screen Voter Menu:

```
--- Voter Menu ---
Voter: Johnson Koh
You have not voted yet.
1. List of Candidates
2. Vote
3. Change Password
0. Logout
Enter your choice: _
```

Choose

- 1 to see the list of candidates.
- 2 to vote for the candidates
- 3 to change the voter's password.
- 0 to Logout, program will return to the main screen.

1. When Voter input "1" to view the List of Candidates

```
--- Voter Menu ---
Voter: Johnson Koh
You have not voted yet.
1. List of Candidates
2. Vote
3. Change Password
0. Logout
Enter your choice: 1

-----LIST OF CANDIDATES-----
=====
No  || Name                || Manifesto
1   || Thompson            || vote me!
2   || Edwin Oo            || One percent Better everyday
3   || Wei Xuan            || Bring UTM glorious again!!!
4   || Jia Bao             || NULL
```

The system will show the list of candidates and with their name and manifesto.

2. When Voter input “2” to Vote

```
--- Voter Menu ---
Voter: Johnson Koh
You have not voted yet.
1. List of Candidates
2. Vote
3. Change Password
0. Logout
Enter your choice: 2

-----LIST OF CANDIDATES-----
=====
No  | |Name                | |Manifesto
1   | |Thompson            | |vote me!
2   | |Edwin Oo            | |One percent Better everyday
3   | |Wei Xuan            | |Bring UTM glorious again!!!
4   | |Jia Bao             | |NULL
Select a candidate to vote (1-4): 2
Are you sure you want to vote for Edwin Oo? (Y/N): y
Vote cast successfully for Edwin Oo.
```

The system will show the list of candidates and ask voters to input the candidate want to vote for, and input “Y” to confirm vote for the candidate, once vote there will be no reverse and no view history due to confidentiality.

3. When Voter input “3” to Change Password

```
--- Voter Menu ---  
Voter: Johnson Koh  
You have already voted.  
1. List of Candidates  
2. Vote  
3. Change Password  
0. Logout  
Enter your choice: 3  
  
=== Change Password ===  
Enter Current Password: nice  
Set New Password: notNice  
Password changed successfully.
```

The system will ask voter to input their current password then set new password, if the current password matches with the system it will update it and show the message “Password change successfully.”

4. When Voter input “0” to Logout

```
--- Voter Menu ---  
Voter: Johnson Koh  
You have already voted.  
1. List of Candidates  
2. Vote  
3. Change Password  
0. Logout  
Enter your choice: 0  
Logging out...  
Press any key to continue . . .
```

The system will clear all history output and return back to main menu.

Candidate User Manual

Main Screen:

```
=== E-Voting System ===
1. Login
2. Register as Voter
3. Display Result
0. Exit
Enter your choice: 1

Enter Username: Trump
Enter Password: word
```

After successful input Voter's username and password, the system will show the output below.

```
--- Candidate Menu ---
Candidate: Mickey Trump
Your current manifesto: Make UTM great again
1. Update Manifesto
2. Change Password
0. Logout
Enter your choice: |
```

Choose

- 1 to update manifesto
- 2 to change the password of the account.
- 0 to Logout, program will return to the main screen.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

1. When the candidate input '1', the system allow candidate to change the manifesto.

```
--- Candidate Menu ---  
Candidate: Mickey Trump  
Your current manifesto: Make UTM great again  
1. Update Manifesto  
2. Change Password  
0. Logout  
Enter your choice: 1  
  
Enter new manifesto: We are the best team!!  
Manifesto updated.  
Note: Any commas have been replaced with semicolons.
```

Enter the new manifesto and it will show in the list of candidates.

2. When the candidate input '2', the system allow candidate to change the password.

```
--- Candidate Menu ---  
Candidate: Mickey Trump  
Your current manifesto: We are the best team!!  
1. Update Manifesto  
2. Change Password  
0. Logout  
Enter your choice: 2  
  
=== Change Password ===  
Enter Current Password: word  
Set New Password: muga  
Password changed successfully.
```

Enter current password and set new password. The password can only be changed if the current password is correct.

The work in this page has been done by: CHING QI YAN Date: 19/6/2025

3. When the candidate input '0', candidates logout from the account.

```
--- Candidate Menu ---  
Candidate: Mickey Trump  
Your current manifesto: We are the best team!!  
1. Update Manifesto  
2. Change Password  
0. Logout  
Enter your choice: 0  
Logging out...  
Press any key to continue . . . |
```

Press any key to return to the main screen.

Appendix

FileManager.hpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
using namespace std;

#ifndef FILEMANAGER_H
#define FILEMANAGER_H

class FileManager{
public:
    static void getFileData(const string& line, string data[], int numFields){
        int a[10], temp=0;
        for(int i=0;i<numFields-1;i++){
            a[i]=line.find(" ",temp);
            data[i]=line.substr(temp,a[i]-temp);
            temp=a[i]+1;
        }
        data[numFields-1]=line.substr(temp);
    }

    static void replaceFile(const string& tempFile, const string& originalFile){
        remove(originalFile.c_str());
        rename(tempFile.c_str(),originalFile.c_str());
    }

    static int countRecords(const string& filename){
        int count=0;
        ifstream file(filename);
        string line;
        getline(file,line); // Skip header
        while(getline(file,line)){
            count++;
        }
        file.close();
        return count;
    }
};

#endif
```

User.hpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include "FileManager.hpp"
using namespace std;

#ifndef USER_H
#define USER_H

class User{
protected:
    string userID;
    string name;
    string password;

public:
    User(string _userID="", string _name="", string _password="")
        : userID(_userID), name(_name), password(_password){}

    virtual void login()=0;
    virtual ~User(){}

    void updatePassword(const string& filename, const string& tempFile, int numFields){
        string oldPassword, newPassword;
        cout<<"\n=== Change Password ===\n";
        cout<<"Enter Current Password: ";
        cin>>oldPassword;
        cout<<"Set New Password: ";
        cin>>newPassword;

        ifstream getFile(filename);
        ofstream changeFile(tempFile);
        string line, data[10];

        getline(getFile,line);
        changeFile<<line<<endl;

        bool found=false;
        while(getline(getFile,line)){
            FileManager::getFileData(line,data,numFields);

            if(userID==data[0] && oldPassword==data[2] && oldPassword!=newPassword){
```

```

        data[2]=newPassword;
        changeFile<<data[0];
        for(int i=1;i<numFields;i++){
            changeFile<<" "<<data[i];
        }
        changeFile<<endl;
        cout<<"Password changed successfully.\n";
        found=true;
    } else if(userID==data[0] && oldPassword==data[2] && oldPassword==newPassword){
        changeFile<<line<<endl;
        cout<<"New password cannot be the same as current password.\n";
        found=true;
    } else if(userID==data[0] && oldPassword!=data[2]){
        changeFile<<line<<endl;
        cout<<"Current password is incorrect.\n";
        found=true;
    } else {
        changeFile<<line<<endl;
    }
}

getFile.close();
changeFile.close();
FileManager::replaceFile(tempFile,filename);

if(!found){
    cout<<"User not found.\n";
}
}

};

#endif

```

Admin.hpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include "FileManager.hpp"
#include "Election.hpp"

#ifndef ADMIN_H
#define ADMIN_H
class Admin : public User{
private:
    Election election;

public:
    Admin(string _userID="", string _name="", string _password="")
        : User(_userID,_name,_password){}

    void login() override{
        cout<<"\nWelcome, Admin: "<<name<<"\n";
        adminMenu();
    }

    static Admin* findAdmin(const string& inputID, const string& inputPassword){
        ifstream inputAdmin("admin.csv");
        string line, adminInfo[3];
        getline(inputAdmin,line); // Skip header

        while(getline(inputAdmin,line)){
            FileManager::getFileData(line,adminInfo,3);
            if(inputID==adminInfo[0] && inputPassword==adminInfo[2]){
                inputAdmin.close();
                return new Admin(adminInfo[0],adminInfo[1],adminInfo[2]);
            }
        }
        inputAdmin.close();
        return nullptr;
    }

    void changeUserPassword(){
        string userID, password;
        cout<<"\n== Change Password for User ==\n";
        cout<<"Enter Username: ";
        cin>>userID;
```

```

cout<<"Set New Password: ";
cin>>password;

// Try to change voter password first
bool found=changePasswordInFile("voters.csv","tempVoters.csv",userID,password,4);

// If not found in voters, try candidates
if(!found){
    found=changePasswordInFile("candidates.csv","tempCandidates.csv",userID,password,5);
}

if(!found){
    cout<<"User not found in voters or candidates.\n";
}
}

bool changePasswordInFile(const string& filename, const string& tempFile, const string& userID, const
string& newPassword, int numFields){
    ifstream getFile(filename);
    ofstream changeFile(tempFile);
    string line, data[10];
    bool found=false;

    getline(getFile,line);
    changeFile<<line<<endl;

    while(getline(getFile,line)){
        FileManager::getFileData(line,data,numFields);

        if(userID==data[0] && newPassword!=data[2]){
            data[2]=newPassword;
            changeFile<<data[0];
            for(int i=1;i<numFields;i++){
                changeFile<<" "<<data[i];
            }
            changeFile<<endl;
            cout<<"Password changed for "<<data[1]<<" successfully.\n";
            found=true;
        }else if(userID==data[0] && newPassword==data[2]){
            changeFile<<line<<endl;
            cout<<"New password cannot be same as old password for "<<data[1]<<"\n";
            found=true;
        }else{
            changeFile<<line<<endl;

```



```

    }
}

getFile.close();
changeFile.close();
FileManager::replaceFile(tempFile,filename);
return found;
}

void getListCandidate(){
    cout<<"\n-----LIST OF CANDIDATES-----\n";
    cout<<"=====\\n";
    cout<<left<<setw(5)<<"No"<<"|"<<setw(20)<<"Name"<<"|"<<"Manifesto\\n";

    ifstream getCandidate("candidates.csv");
    string line, candidate[5];
    int count=1;

    getline(getCandidate,line); // Skip header
    while(getline(getCandidate,line)){
        FileManager::getFileData(line,candidate,5);
        cout<<left<<setw(5)<<count++<<"|"<<setw(20)
            <<candidate[1]<<"|"<<candidate[3]<<endl;
    }
    getCandidate.close();
}

void addCandidate(){

    if(election.getElectionData()){
        cout<<"There is ongoing election, you are not able to add candidate.\\n";
        return;
    }

    char ctn;
    do{
        string userID, name, password;
        cout<<"\\n=== Add New Candidate ===\\n";
        cout<<"Enter Candidate Username: ";
        cin>>userID;
        cin.ignore();
        cout<<"Enter Candidate Full Name: ";
        getline(cin,name);
        cout<<"Set Candidate Password: ";

```

```

cin>>password;

ifstream checkCandidate("candidates.csv");
string line, candidate[5];
bool exists=false;

getline(checkCandidate,line);
while(getline(checkCandidate,line)){
    FileManager::getFileData(line,candidate,5);
    if(userID==candidate[0]){
        exists=true;
        break;
    }
}
checkCandidate.close();

if(exists){
    cout<<"Candidate with this Username already exists.\n";
}else{
    ofstream addCandidate("candidates.csv",ios::app);
    addCandidate<<userID<<","<<name<<","<<password<<","<<NULL,0"<<endl;
    addCandidate.close();

    cout<<"Candidate '"<<name<<"' added successfully.\n";
    cout<<"Note: Candidate can login to add their manifesto.\n";
}

getListCandidate();

cout<<"\nDo you want to add another candidate? (Y/N): ";
cin>>ctn;

} while(ctn=='Y' || ctn=='y');
}

int readNumOfCandidates(){
return FileManager::countRecords("candidates.csv");
}

void addVoter(){
char continueAdd;
do{
    string userID, name;
    cout<<"\n=== Add New Voter ===\n";

```

```

    cout<<"Enter Voter Username: ";
    cin>>userID;
    cin.ignore();
    cout<<"Enter Voter Full Name: ";
    getline(cin,name);

    ifstream checkVoter("voters.csv");
    string line, voter[4];
    bool exists=false;

    getline(checkVoter,line);
    while(getline(checkVoter,line)){
        FileManager::getFileData(line,voter,4);
        if(userID==voter[0]){
            exists=true;
            break;
        }
    }
    checkVoter.close();

    if(exists){
        cout<<"Voter with this Username already exists.\n";
    }else{
        ofstream addVoter("voters.csv",ios::app);
        addVoter<<userID<<","<<name<<","<<"False"<<endl;
        addVoter.close();

        cout<<"Voter '"<<name<<"' added successfully.\n";
        cout<<"Note: Voter needs to register to set their password.\n";
    }

    cout<<"\nDo you want to add another voter? (Y/N): ";
    cin>>continueAdd;

    } while(continueAdd=='Y' || continueAdd=='y');
}

void adminMenu(){
    char choice;
    do{
        cout<<"\n--- Admin Menu ---\n";
        cout<<"1. Change Password for user\n";
        cout<<"2. List of Candidates\n";
        cout<<"3. Add Candidate\n";
    } while(choice != 'q');
}

```

```

cout<<"4. Add Voter\n";
cout<<"5. Election Status\n";
cout<<"6. View Election History\n";
cout<<"0. Logout\n";
cout<<"Enter your choice: ";
cin>>choice;

switch(choice){
    case '1':
        changeUserPassword();
        break;
    case '2':
        getListCandidate();
        break;
    case '3':
        addCandidate();
        break;
    case '4':
        addVoter();
        break;
    case '5':
        election.startElection();
        break;
    case '6':
        election.viewElectionHistory();
        break;
    case '0':
        cout<<"Logging out...\n";
        break;
    default:
        cout<<"Invalid choice.\n";
}
}while(choice!='0');
}
};
#endif

```

Voter.hpp

```

#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include "FileManager.hpp"

#ifndef VOTER_H
#define VOTER_H
class Voter : public User{
private:
    bool hasVoted;
    bool needsUpdate;
    Admin *admin;

public:
    Voter(string _userID="", string _name="", string _password="", bool _hasVoted=false)
        : User(_userID, _name, _password), hasVoted(_hasVoted), needsUpdate(false), admin(nullptr){}

    void login() override{
        voterMenu();
    }

    ~Voter(){
        delete admin;
    }

    void setAdmin(Admin* adminPtr){
        admin=adminPtr;
    }

    static Voter* findVoter(const string& inputID, const string& inputPassword){
        ifstream inputVoters("voters.csv");
        string line, voters[4];
        getline(inputVoters,line); // Skip header

        while(getline(inputVoters,line)){
            FileManager::getFileData(line,voters,4);
            if(inputID==voters[0] && inputPassword==voters[2]){
                bool _hasVoted=(voters[3]=="True");
                inputVoters.close();
                return new Voter(voters[0],voters[1],voters[2],_hasVoted);
            }
        }
    }
}

```

```

inputVoters.close();
return nullptr;
}

void changePassword(){
    updatePassword("voters.csv","tempVoters.csv",4);
}

static void registerVoter(){
    string userID, password, name;
    cout<<"\n=== Voter Registration ===\n";
    cout<<"Enter your Username: ";
    cin>>userID;
    cin.ignore();
    cout<<"Enter your Full Name: ";
    getline(cin,name);
    cout<<"Set your Password: ";
    cin>>password;

    ifstream getVoter("voters.csv");
    ofstream regVoter("temp.csv");
    string line, voters[4];
    bool found=false;

    getline(getVoter,line);
    regVoter<<line<<endl;

    while(getline(getVoter,line)){
        FileManager::getFileData(line,voters,4);

        if(userID==voters[0] && name==voters[1] && voters[2]==""){
            regVoter<<userID<<","<<name<<","<<password<<","<<"False"<<endl;
            found=true;
        }else{
            regVoter<<line<<endl;
        }
    }

    getVoter.close();
    regVoter.close();
    FileManager::replaceFile("temp.csv","voters.csv");
    if(found){
        cout<<"Registration successful!\n";
    }else{

```

```

        cout<<"Registration failed. Check your Username and Name.\n";
    }
}

void vote(){
    Election election;

    if(!election.getElectionData()){
        cout<<"No ongoing election. You cannot vote at this time.\n";
        return;
    }

    if(hasVoted){
        cout<<"You have already voted.\n";
        return;
    }

    int numCandidates=admin->readNumOfCandidates();
    int choice;
    admin->getListCandidate();

    while(true){
        try{
            cout<<"Select a candidate to vote (1- "<<numCandidates<<"): ";
            string input;
            cin>>input;

            choice=stoi(input); // This throws exception if input is not a number

            if(choice<1 || choice>numCandidates){
                cout<<"Invalid choice. Please enter a number between 1 and "<<numCandidates<<".\n";
                continue;
            }

            break; // Valid input, exit loop

        } catch(exception& e){
            cout<<"Invalid input. Please enter a valid number.\n";
        }
    }

    // Cast vote
    ifstream getCandidates("candidates.csv");
    ofstream voteCandidates("tempCandidates.csv");

```

```

string line, candidate[5];
int currentCandidate=1;

getline(getCandidates,line);
voteCandidates<<line<<endl;

while(getline(getCandidates,line)){
    FileManager::getFileData(line,candidate,5);

    if(choice==currentCandidate){
        char confirm;
        cout<<"Are you sure you want to vote for "<<candidate[1]<<"? (Y/N): ";
        cin>>confirm;

        if(confirm!='y' && confirm!='Y'){
            cout<<"Vote cancelled.\n";
            getCandidates.close();
            voteCandidates.close();
            remove("tempCandidates.csv");
            return;
        }

        int newVoteCount=stoi(candidate[4])+1;
        voteCandidates<<candidate[0]<<","<<candidate[1]<<","<<
            <<candidate[2]<<","<<candidate[3]<<","<<
            <<newVoteCount<<endl;

        cout<<"Vote cast successfully for "<<candidate[1]<<".\n";
        hasVoted=true;
        needsUpdate=true;
    }else{
        voteCandidates<<line<<endl;
    }
    currentCandidate++;
}

getCandidates.close();
voteCandidates.close();
FileManager::replaceFile("tempCandidates.csv","candidates.csv");
reloadData();
}

void reloadData(){
    ifstream getVoter("voters.csv");
    ofstream changeVoter("tempVoters.csv");

```



```

string line, voter[4];

getline(getVoter,line);
changeVoter<<line<<endl;

while(getline(getVoter,line)){
    FileManager::getFileData(line,voter,4);

    if(userID==voter[0]){
        if(needsUpdate){
            changeVoter<<voter[0]<<","<<voter[1]<<","<<
                <<voter[2]<<","<<True"<<endl;
            needsUpdate=false;
        }else{
            changeVoter<<line<<endl;
            hasVoted=(voter[3]=="True");
        }
    }else{
        changeVoter<<line<<endl;
    }
}

getVoter.close();
changeVoter.close();
FileManager::replaceFile("tempVoters.csv","voters.csv");
}

void voterMenu(){
    char choice;
    do{
        reloadData();
        cout<<"\n--- Voter Menu ---\n";
        cout<<"Voter: "<<name<<"\n";
        cout<<(hasVoted ? "You have already voted.\n" : "You have not voted yet.\n");
        cout<<"1. List of Candidates\n";
        cout<<"2. Vote\n";
        cout<<"3. Change Password\n";
        cout<<"0. Logout\n";
        cout<<"Enter your choice: ";
        cin>>choice;

        switch(choice){
            case '1':
                admin->getListCandidate();
                break;

```

```
        case '2':
            vote();
            break;
        case '3':
            changePassword();
            break;
        case '0':
            cout<<"Logging out...\n";
            break;
        default:
            cout<<"Invalid choice.\n";
    }
} while(choice!='0');
};
#endif
```

Candidate.hpp

```

#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include "FileManager.hpp"

#ifndef CANDIDATE_H
#define CANDIDATE_H
class Candidate : public User{
private:
    string manifesto;
    int voteCount;
    bool needsUpdate;

public:
    Candidate(string _userID="", string _name="", string _password="",
               string _manifesto="NULL", int _votes=0)
        : User(_userID, _name, _password), manifesto(_manifesto),
          voteCount(_votes), needsUpdate(false){}

    void login() override{
        candidateMenu();
    }

    static Candidate* findCandidate(const string& inputID, const string& inputPassword){
        ifstream inputCandidates("candidates.csv");
        string line, candidates[5];
        getline(inputCandidates, line); // Skip header

        while(getline(inputCandidates, line)){
            FileManager::getFileData(line, candidates, 5);
            if(inputID==candidates[0] && inputPassword==candidates[2]){
                inputCandidates.close();
                return new Candidate(candidates[0], candidates[1], candidates[2],
                                     candidates[3], stoi(candidates[4]));
            }
        }
        inputCandidates.close();
        return nullptr;
    }

    void updateManifesto(){
        cout<<"\nEnter new manifesto: ";
    }

```

```

        cin.ignore();
        getline(cin,manifesto);

        for(int i=0;i<manifesto.length();i++){
            if(manifesto[i]==';'){
                manifesto[i]='.';
            }
        }

        cout<<"Manifesto updated.\n";
        cout<<"Note: Any commas have been replaced with semicolons.\n";
        needsUpdate=true;
        reloadData();
    }

    void changePassword(){
        updatePassword("candidates.csv","tempCandidates.csv",5);
    }

    void reloadData(){
        ifstream getCandidate("candidates.csv");
        ofstream changeCandidate("tempCandidates.csv");
        string line, candidate[5];

        getline(getCandidate,line);
        changeCandidate<<line<<endl;

        while(getline(getCandidate,line)){
            FileManager::getFileData(line,candidate,5);

            if(userID==candidate[0]){
                if(needsUpdate){
                    changeCandidate<<candidate[0]<<" "<<candidate[1]<<" "<<
                        <<candidate[2]<<" "<<manifesto<<" "<<
                        <<candidate[4]<<endl;
                    needsUpdate=false;
                }else{
                    changeCandidate<<line<<endl;
                    manifesto=candidate[3];
                }
            }else{
                changeCandidate<<line<<endl;
            }
        }
    }

```

```

getCandidate.close();
changeCandidate.close();
FileManager::replaceFile("tempCandidates.csv","candidates.csv");
}

void candidateMenu(){
    char choice;
    do{
        reloadData();
        cout<<"\n--- Candidate Menu ---\n";
        cout<<"Candidate: "<<name<<"\n";
        cout<<"Your current manifesto: "<<manifesto<<"\n";
        cout<<"1. Update Manifesto\n";
        cout<<"2. Change Password\n";
        cout<<"0. Logout\n";
        cout<<"Enter your choice: ";
        cin>>choice;

        switch(choice){
            case '1':
                updateManifesto();
                break;
            case '2':
                changePassword();
                break;
            case '0':
                cout<<"Logging out...\n";
                break;
            default:
                cout<<"Invalid choice.\n";
        }
    }while(choice!='0');
}

};

#endif

```

Election.hpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include "FileManager.hpp"

using namespace std;

#ifndef ELECTION_H
#define ELECTION_H

class Election{
private:
    string electionID;
    string title;
    bool status;

public:
    Election(string _electionID="000", string _title="", bool _status=false)
        : electionID(_electionID), title(_title), status(_status){}

    void clearVotingData(){ //After election done, the system will auto reset their data
        clearCandidateVotes();
        clearVoterStatus();
    }

    void clearCandidateVotes(){
        ifstream getCandidate("candidates.csv");
        ofstream changeCandidate("tempCandidates.csv");
        string line, candidate[5];

        getline(getCandidate,line);
        changeCandidate<<line<<endl;

        while(getline(getCandidate,line)){
            FileManager::getFileData(line,candidate,5);
            changeCandidate<<candidate[0]<<","<<candidate[1]<<","<<
                <<candidate[2]<<","<<candidate[3]<<","<<0<<endl;
        }

        getCandidate.close();
        changeCandidate.close();
        FileManager::replaceFile("tempCandidates.csv","candidates.csv");
    }
};
```

```

}

void clearVoterStatus(){
    ifstream getVoter("voters.csv");
    ofstream changeVoter("tempVoters.csv");
    string line, voter[4];

    getline(getVoter,line);
    changeVoter<<line<<endl;

    while(getline(getVoter,line)){
        FileManager::getFileData(line,voter,4);
        changeVoter<<voter[0]<<","<<voter[1]<<","<<voter[2]<<","False"<<endl;
    }

    getVoter.close();
    changeVoter.close();
    FileManager::replaceFile("tempVoters.csv","voters.csv");
}

bool getElectionData() { //get election status
    ifstream getElection("election.csv");
    string line, data[3];

    getline(getElection,line); // Skip header
    while(getline(getElection,line)){
        FileManager::getFileData(line,data,3);
        if(data[2]=="Ongoing"){
            electionID=data[0];
            title=data[1];
            getElection.close();
            return true;
        }
        electionID=data[0];
    }
    getElection.close();
    return false;
}

void startElection(){
    cout<<"\n--- Election Status ---\n";
    if(getElectionData()){
        cout<<"You cannot start a new Election.\nThere is already an ongoing election!\n";
    }
}

```

```

        cout<<"Title: "<<title<<"\n";
        closeElection();
        return;
    }

    cout<<"There is no ongoing Election, you may start a new election.\n";
    string _title;
    char confirm;
    cout<<"Enter the Election title: ";
    cin.ignore();
    getline(cin, _title);
    cout<<"Enter 'Y' to confirm: ";
    cin>>confirm;

    if(confirm=='Y' || confirm=='y'){
        int temp_electionID=stoi(electionID)+1;
        electionID=to_string(temp_electionID);
        temp_electionID=3-electionID.length();
        if(temp_electionID>0){
            electionID.insert(0,"000",temp_electionID);
        }
        title=_title;
        status=true;
        cout<<"Election '"<<title<<"' has started.\n";

        ofstream inputElection("election.csv",ios::app); //app stand for append
        inputElection<<electionID<<","<<title<<","<<"Ongoing"<<endl;
        inputElection.close();
    }else{
        cout<<"Election not started.\n";
    }
}

void closeElection(){
    if(!getElectionData()){
        cout<<"There is no ongoing Election to be closed!!!\n";
        return; //cannot print result
    }
}

char confirm;
ifstream getElection("election.csv");
ofstream changeElection("tempElection.csv");
string line, data[3];

```



```

getline(getElection,line);
changeElection<<line<<endl;

while(getline(getElection,line)){
    FileManager::getFileData(line,data,3);

    if(data[2]=="Ongoing"){
        cout<<"Do you want to close this Election "<<data[1]<<"?"<<"\n";
        cout<<"Enter 'Y' to confirm end election: ";
        cin>>confirm;

        if(confirm=='y' || confirm=='Y'){
            changeElection<<data[0]<<" "<<data[1]<<"Ended"<<endl;
            cout<<"Election "<<data[1]<<" has ended successfully.\n";

            getElection.close();
            changeElection.close();
            FileManager::replaceFile("tempElection.csv","election.csv");

            generateResult(); //Generate result file first
            clearVotingData(); //then clear voting data
            return;
        }else{
            cout<<"Return back.\n";
            getElection.close();
            changeElection.close();
            remove("tempElection.csv");
            return;
        }
    }else{
        changeElection<<line<<endl;
    }
}

getElection.close();
changeElection.close();
FileManager::replaceFile("tempElection.csv","election.csv");
}

string getElectionID(){
    string line, temp_line;
    ifstream getElection("election.csv");
    getline(getElection,line);

```

```

while(getline(getElection,line)){
    temp_line=line;
}
getElection.close();

string data[3];
FileManager::getFileData(temp_line,data,3);
return electionID=data[0];
}

void generateResult(){
string filename="Result_"+getElectionID()+".txt";
ofstream outputResult(filename);
ifstream getCandidate("candidates.csv");

int numCandidates=FileManager::countRecords("candidates.csv");
string names[numCandidates];
int votes[numCandidates];
int count=0;
string line;

getline(getCandidate,line); // Skip header
while(getline(getCandidate,line)){
    string data[5];
    FileManager::getFileData(line,data,5);
    names[count]=data[1];
    votes[count]=stoi(data[4]);
    count++;
}

// Sort by votes (bubble sort)
for(int i=0;i<count-1;i++){
    for(int j=0;j<count-1-i;j++){
        if(votes[j]<votes[j+1]){
            swap(votes[j],votes[j+1]);
            swap(names[j],names[j+1]);
        }
    }
}

string winners="" + names[0] + "";
for(int i=1;i<count && votes[i]==votes[0];i++){
    winners+=" and " + names[i] + "";
}

```

```

outputResult<<"Title: "<<title<<endl;
outputResult<<"Highest votes of candidate: "<<winners<<endl;
outputResult<<left<<setw(7)<<"Ranking"<<"||"<<setw(20)
        <<"Candidates' Name"<<"||Number of Votes\n";

int currentRank=1;
for(int i=0;i<count;i++){

    if(i>0 && (votes[i]!=votes[i-1])){
        currentRank=i+1; // Set rank to current position + 1
    }

    outputResult<<left<<setw(7)<<currentRank<<"||"<<setw(20)
        <<names[i]<<"||"<<votes[i]<<endl;
}

getCandidate.close();
outputResult.close();

displayResult(filename);
}

void displayResult(const string& filename){
    cout<<"\n=== ELECTION RESULT ===\n";
    ifstream display(filename);
    string line;
    while(getline(display,line)){
        cout<<line<<endl;
    }
    display.close();
}

void displayResult(){
    // First check if there's an ongoing election
    if(getElectionData()){
        cout<<"There is an ongoing election: "<<title<<"\n";
        cout<<"Showing result of previous election...\n\n";
    }

    string electionID=getElectionID();

    // If there's ongoing election, get the previous election ID
    if(getElectionData()){

```

```

        int prevID=stoi(electionID)-1;
        if(prevID>0){
            electionID=to_string(prevID);
            prevID=3-electionID.length();
            if(prevID>0){
                electionID.insert(0,"000",prevID);
            }
        }else{
            cout<<"No previous election found.\n";
            return;
        }
    }

    string filename="Result_"+electionID+".txt";

    ifstream display(filename);
    if(!display.is_open()){
        cout<<"No result file found. Please ensure an election has been completed.\n";
        return;
    }

    cout<<"\n=== ELECTION RESULT ===\n";
    string line;
    while(getline(display,line)){
        cout<<line<<endl;
    }
    display.close();
}

void viewElectionHistory(){
    cout<<"\n=== Election History ===\n";
    cout<<left<<setw(5)<<"No"<<"|"<<setw(10)<<"Election ID"
        <<"|"<<setw(30)<<"Title"<<"|Status\n";
    cout<<"===== \n";

    ifstream getElection("election.csv");
    string line, data[3];
    int count=1;

    getline(getElection,line); // Skip header
    while(getline(getElection,line)){
        FileManager::getFileData(line,data,3);
        cout<<left<<setw(5)<<count++<<"|"<<setw(10)<<data[0]
            <<"|"<<setw(30)<<data[1]<<"|"<<data[2]<<endl;
    }
}

```

```

    }
    getElection.close();

    if(count==1){
        cout<<"No election history found.\n";
        return;
    }

    string selectedID;
    cout<<"\nEnter Election ID to view result (or 0 to return): ";
    cin>>selectedID;

    if(selectedID=="0") return;

    string filename="Result_"+selectedID+".txt";
    ifstream display(filename);
    if(!display.is_open()){
        cout<<"Result file not found for Election ID: "<<selectedID<<"\n";
        return;
    }

    cout<<"\n=== ELECTION RESULT ===\n";
    string resultLine;
    while(getline(display,resultLine)){
        cout<<resultLine<<endl;
    }
    display.close();
}
};
#endif

```

main.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

#include "FileManager.hpp"
#include "Election.hpp"
#include "User.hpp"
#include "Admin.hpp"
#include "Candidate.hpp"
#include "Voter.hpp"

using namespace std;

// Main function
int main(){
    char choice;
    do{
        cout<<"=== E-Voting System ===\n";
        cout<<"1. Login\n";
        cout<<"2. Register as Voter\n";
        cout<<"3. Display Result\n";
        cout<<"0. Exit\n";
        cout<<"Enter your choice: ";
        cin>>choice;

        string id, pwd;
        User* user=nullptr;

        switch(choice){
            case '1':
                cout<<"\nEnter Username: ";
                cin>>id;
                cout<<"Enter Password: ";
                cin>>pwd;

                if(user=Admin::findAdmin(id,pwd)){
                    user->login();
                    delete user;
                }else if(user=Voter::findVoter(id,pwd)){
                    Voter* voter=static_cast<Voter*>(user);
                    voter->setAdmin(new Admin());
                    user->login();
                }
            }
        }
```

```

        delete user;
    }else if(user=Candidate::findCandidate(id,pwd)){
        user->login();
        delete user;
    }else{
        cout<<"Invalid login credentials.\n";
    }
    break;

case '2':
    Voter::registerVoter();
    break;

case '3':{
    Election election;
    election.displayResult();
    break;
}

case '0':
    cout<<"\nExiting system...\n";
    break;
default:
    cout<<"Invalid choice. Try again.\n";
}
system("pause");
system("cls");
}while(choice!='0');

return 0;
}

```

admin.csv

Username	Name	Password
Admin123	Admin John	AdminPass
a	a	a

candidate.csv

UserID	Name	Password	Manifesto	Votes
testingla	Thompson	abc	vote me!	0
candi	Edwin Oo	pass	One percent Better everyday	1
wlao	Wei Xuan	abc	Bring UTM glorious again!!!	0
candi1	Jia Bao	candiPass	NULL	0

election.csv

ElectionID	Title	Status
001	test1	Ended
002	asdfg	Ended
003	Leader of FC	Ongoing

voters.csv

UserID	Name	Password	HasVoted
kwei	Keng Wei	pass	False
qiyan	Ching Qi Yan	qy	False
oo	Edwin	oo	False
voter	Sunny	pass	False
voter2	Jason	pass	False
john	Johnson Koh	notNice	True